

Recursive and Iterative Solutions

DAEN 300

Alexander Abuabara

Fall 2026



- ▶ Predict what a loop and a recursive function return.
- ▶ Explain how a recursive program knows when to stop.
- ▶ Recognize tail and non-tail recursive calls.
- ▶ Compare solutions using tests and measured times.

You should already know functions, lists, indexing, `if`, and `for/while` loops.

Remember: `return` sends a value to the caller.

`print` displays a value on the screen.

Totaling survey file counts

A school survey produces three data files. Their row counts are:

```
counts = [12, 8, 5]  
# The combined total should be 25.
```

Iteration repeats steps with a loop.

Recursion repeats work through self-calls on smaller problems.

A **contract** states the allowed input and expected result.

Here: take a flat list of integers and return its total.

Return 0 for an empty list. Keep the input unchanged.

A loop keeps a running total

```
def sum_iterative(values):  
    total = 0  
    for value in values:  
        total += value  
    return total  
  
sum_iterative(counts) # 25
```

The running total is 0, then 12, 20, and 25.

After every step, it includes all values processed so far.

Predict: What does the function return for []?

The recursive summation plan

At a position called `index`, the unfinished task is:
total the values starting at that position.

1. **Stop:** if the position reaches the list length, return 0. This is the **base case**.
2. **Reduce:** ask the same function to total the later values, starting at `index + 1`.
3. **Combine:** add the current value to that answer.

The unchecked part loses one item each time.
This progress makes the base case reachable.

A recursive total

```
def sum_recursive(values, index=0):  
    if index == len(values):  
        return 0  
    later_total = sum_recursive(values, index + 1)  
    return values[index] + later_total  
  
sum_recursive(counts) # 25
```

The first call starts at position 0.

Each call shares the original list and advances the position. Using a position avoids making a new tail list with `values[1:]`.

The caller waits for `later_total`, then adds its value.

Calls wait, then return

For [12, 8, 5]:

Position	This call waits to calculate	Result
0	12 + answer from position 1	25
1	8 + answer from position 2	13
2	5 + answer from position 3	5
3	Base case	0

Calls start at positions 0, 1, 2, 3.

Results return in the order 0, 5, 13, 25.

Python's **call stack** remembers the paused calls and their local variables. The latest call finishes first.

Non-tail recursion

A recursive call is **non-tail** when its caller still has a calculation to do after the call returns.

In our recursive total, the call at position 0:

1. Requests the total of [8, 5].
2. Waits for the answer 13.
3. Adds 12 and returns 25.

The addition happens **after** the recursive call.

Even return $x + f(\dots)$ needs an addition after f returns.

Being on the final source line is not enough.

Tail recursion carries the result so far

```
def sum_tail(values, index=0, total=0):  
    if index == len(values):  
        return total  
    return sum_tail(  
        values, index + 1, total + values[index]  
    )  
  
sum_tail(counts) # 25
```

total is an **accumulator**: the result so far.

Python calculates the new arguments before making the call.

When it returns, the caller passes its answer back unchanged. This is a **tail call**.

Tracing the accumulator

Each call receives the values shown here:

index	total	Action
0	0	Pass position 1 and total 12
1	12	Pass position 2 and total 20
2	20	Pass position 3 and total 25
3	25	Base case returns 25

This version adds the values while moving forward.

The earlier version adds them while calls return.

Both give 25. In Python, the tail version still has waiting calls. It passes 25 back through each one.

A loop with an index and a total

```
def sum_while(values):  
    index = 0  
    total = 0  
    while index < len(values):  
        total += values[index]  
        index += 1  
    return total
```

The tail version passes a new `index` and `total`.

The loop updates those variables inside one call.

Both move forward through the same positions. This is one way to translate a tail-recursive program into iteration.

What makes a call a tail call?

In our totals	Non-tail version	Tail version
After the self-call returns	Add a value	Return unchanged
Result so far	Waiting additions	Running total
Base case	Return 0	Return total

Both functions check their base case first. The location of the base case does not define tail recursion.

Check the work after the call, rather than only the position of the call in the source code.

Python still has a depth limit

```
import sys
print(sys.getrecursionlimit())
```

Some languages reuse a call frame for a tail call.

CPython does not optimize Python tail calls this way.

For our list totals, both recursive versions add one active call per item, plus the base case. Both can reach `RecursionError` on a valid long list.

The loop keeps one call active. In the lab, keep the configured recursion limit and compare the outcomes.

Three mistakes to check

1. **Missing stopping case.** Does an empty input return a useful answer before indexing?
2. **Missing progress.** Passing `index` again asks for the same work. Pass `index + 1`.
3. **Missing return.** Calling a function and returning its answer are separate actions. Forward the answer when the caller needs it.

Use `[]`, one item, and a few items to check these cases. Our examples read shared inputs without changing them. This keeps the comparison easier to reason about.

Practice with a word game

A **palindrome** reads the same in both directions:

```
"level" # True  
"1221" # True  
"python" # False
```

Compare the outer pair, then move inward. This gives us a second example of a smaller remaining problem.

Contract: compare the exact string.

Case, spaces, and punctuation count.

"Level" is False. Empty and one-character strings are True.

Palindrome with iteration

```
def palindrome_iterative(text):  
    left = 0  
    right = len(text) - 1  
    while left < right:  
        if text[left] != text[right]:  
            return False  
        left += 1  
        right -= 1  
    return True
```

A mismatch stops the check immediately.

If the positions meet or cross, all required pairs matched. No separate reversal of the string is needed.

Palindrome with recursion

```
def palindrome_recursive(text, left=0, right=None):  
    if right is None:  
        right = len(text) - 1  
    if left >= right:  
        return True  
    if text[left] != text[right]:  
        return False  
    return palindrome_recursive(  
        text, left + 1, right - 1  
    )
```

None lets the initial call set the right position.

The unchecked region shrinks by two characters.

Predict: Is this self-call tail or non-tail? Why?

Why nested data changes the task

A folder may contain data files or smaller groups of files. Represent their row counts with nested lists:

```
batches = [120, [80, 45], [], [30, [25]]]  
# Total: 300 rows
```

A fixed pair of loops handles only a fixed number of levels. Here, the number of levels can vary.

Each smaller group has the same form as the whole group.

Recursion can follow that structure.

Contract: an integer or a finite nested list of integers. Empty lists contribute 0. No cycles or other data types. Keep the input unchanged.

A recursive total for nested lists

```
def nested_sum_recursive(data):  
    if isinstance(data, int):  
        return data  
    total = 0  
    for item in data:  
        total += nested_sum_recursive(item)  
    return total  
  
nested_sum_recursive(batches) # 300
```

An integer returns directly. An empty list returns 0 because its loop has no items.

The loop combines the child results. A recursive function can contain a loop. This self-call is non-tail because the caller adds its result and may process more children.

Iteration keeps a list of pending work

```
def nested_sum_iterative(data):  
    total = 0  
    stack = [data]  
    while stack:  
        item = stack.pop()  
        if isinstance(item, list):  
            stack.extend(item)  
        else:  
            total += item  
    return total
```

pop() takes the last pending item. extend() adds children. This is an explicit **stack** of work.

For [8, [4, 3]], it adds 3, then 4, then 8. Integer addition gives the same total, 15.

Choosing a solution

For a flat list with n items, in CPython:

Our version	Additions	Active calls at most
Loop	n	1
Non-tail total	n	$n + 1$
Tail total	n	$n + 1$

For flat counts, a loop or Python's built-in `sum` is a direct choice. For a small nested input, recursion may be easy to follow. For unpredictable depth, an explicit stack avoids recursive calls, while still storing pending work.

Readability and measured time both help you choose.

Lab Assignment

- A:** Trace three supplied totals and add two tests.
- B:** Write two palindrome functions and add a test.
- C:** Write two nested totals and trace the explicit stack.
- D:** Measure three totals and observe their depth behavior.
- E:** Choose solutions and explain your evidence.

First test ordinary and boundary inputs against expected answers.
Two programs agreeing can still both be wrong.
Complete the notebook's named answer cells. Then restart the kernel and run every cell before submission.

Timing comparison

```
from timeit import repeat

values = [1] * 100
samples = repeat(lambda: sum_iterative(values),
                  number=200, repeat=5)
best_us = min(samples) / 200 * 1_000_000
print(best_us)
```

The small `lambda` makes the call we want to measure. Measure all three totals on the same input. Build data outside the timer. Keep printing and tracing outside it too. Report microseconds per call for several input sizes. Record your Python version. Use your results rather than assuming that one version must always be faster.